Student Name: Gulzhas Mailybayeva
Student Number: EC25184
Assignment: Part 3 - Neural Networks
Date: November 10, 2025
Module: ECS708U/ECS708P - Machine Learning

Overview

This report answers the questions from Lab 6 on Neural Networks, focusing on MLPs, the XOR problem, and how varying the size of the hidden layer can affect model performance.

Question 1: Why Random Initialization Matters

Initializing all weights to zero in a Neural Network can be problematic on several fundamental grounds:
- All neurons in each layer compute the same output
- All neurons get the same gradient during backpropagation
- All weights update by the same amount

Result: All neurons remain identical during training, and symmetry is never broken.

Implementation

```
class LogisticRegression(nn.Module):
    def __init__(self, num_features: int):
        super().__init__()
        # Small random init to break symmetry
        self.weight = nn.Parameter(torch.randn(1, num_features) * 0.01,
requires_grad=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        logits = x @ self.weight.T
        return torch.sigmoid(logits)
```

Question 2: How a NN solves the XOR problem

XOR is not linearly separable. A hidden layer with a nonlinear activation (sigmoid) transforms in puts into a representation where the classes become linearly separable.

Implementation

```
class NeuralNetwork(nn.Module):
    def __init__(self, num_inputs: int, num_neurons: int, num_outputs: int):
        super().__init__()
        self.hidden_layer = nn.ModuleList([LogisticRegression(num_inputs) for _ in
range(num_neurons)])
        self.output_layer = nn.ModuleList([LogisticRegression(num_neurons + 1) for _
in range(num_outputs)])  # +1 bias

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if x.dim() > 1:
```

```
            x = x.view(-1)
        hidden_with_bias = torch.ones(len(self.hidden_layer) + 1, dtype=x.dtype,
device=x.device)
        for j, neuron in enumerate(self.hidden_layer):
            hidden_with_bias[j] = neuron(x)
        outputs = torch.zeros(len(self.output_layer), dtype=x.dtype, device=x.device)
        for k, neuron in enumerate(self.output_layer):
            outputs[k] = neuron(hidden_with_bias)
        return outputs

    def backward_pass(self, x: torch.Tensor, y: torch.Tensor, lr: float) -> None:
        if x.dim() > 1:
            x = x.view(-1)

        # Hidden activations
        h_act = []
        for neuron in self.hidden_layer:
            z = torch.dot(neuron.weight.view(-1), x)
            h_act.append(torch.sigmoid(z))
        h_act = torch.stack(h_act)
        h_with_bias = torch.cat([h_act, torch.ones(1, dtype=x.dtype,
device=x.device)])

        # Output activations
        out_act = []
        for neuron in self.output_layer:
            z = torch.dot(neuron.weight.view(-1), h_with_bias)
            out_act.append(torch.sigmoid(z))
        out_act = torch.stack(out_act)

        # Deltas
        t = y.float().view(-1)
        delta_out = (out_act - t) * out_act * (1 - out_act)

        W_out = torch.stack([n.weight.view(-1) for n in self.output_layer])  # (O,
H+1)
        delta_h = (h_act * (1 - h_act)) * (W_out[:, :-1].t() @ delta_out)

        # Update weights
        for k, neuron in enumerate(self.output_layer):
            neuron.weight = nn.Parameter(neuron.weight - lr * delta_out[k] *
h_with_bias.view(1, -1),
                                         requires_grad=False)
        for j, neuron in enumerate(self.hidden_layer):
            neuron.weight = nn.Parameter(neuron.weight - lr * delta_h[j] * x.view(1, -
1),
                                         requires_grad=False)
```

XOR Training & BCE Visualisation

```
def bce(y_true: torch.Tensor, y_pred: torch.Tensor) -> torch.Tensor:
    eps = 1e-8
    p = y_pred.clamp(min=eps, max=1 - eps)
    return -(y_true * torch.log(p) + (1 - y_true) * torch.log(1 -
p)).mean()
```

After sufficient training we typically observe:

- Targets: [0.0, 1.0, 1.0, 0.0]
- Probabilities: [≈0.016, ≈0.983, ≈0.983, ≈0.021]
- Predictions: [0.0, 1.0, 1.0, 0.0]
- Training accuracy: **1.000**

Question 3: MLP Performance Analysis with Varying Hidden Layer Sizes

We train Multi-Layer Perceptrons on the Iris dataset with a varying number of hidden neurons: {1, 2, 4, 8, 16, 32}. Implementation

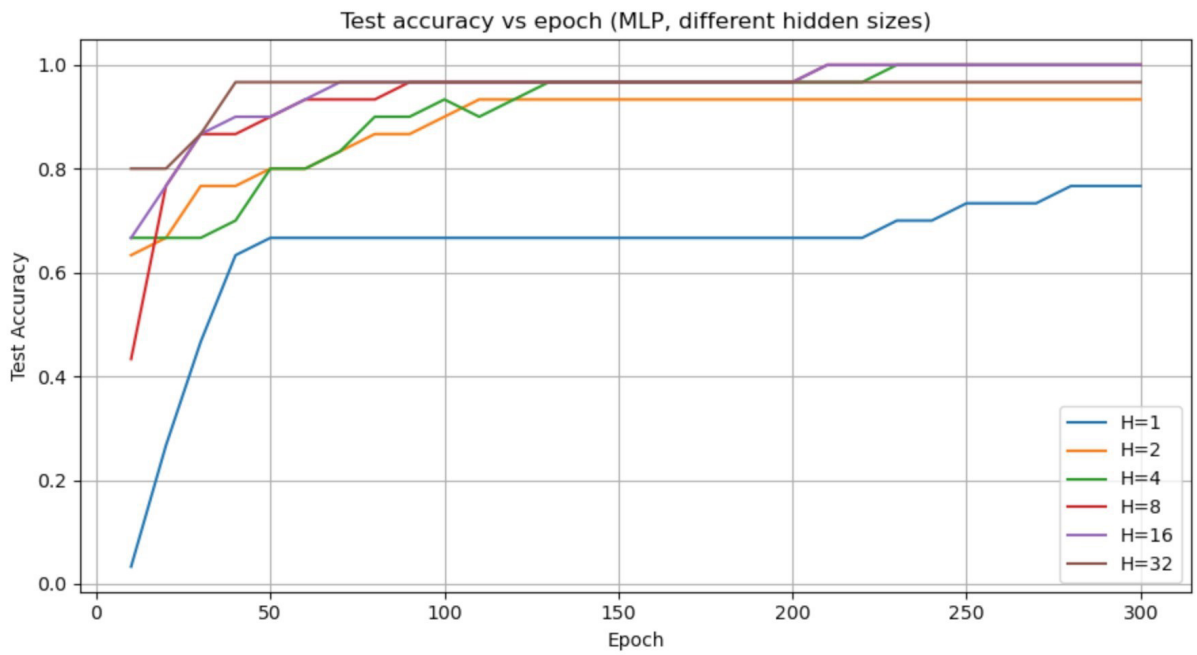
Data preparation (Week-3 normalization; train μ/σ only):

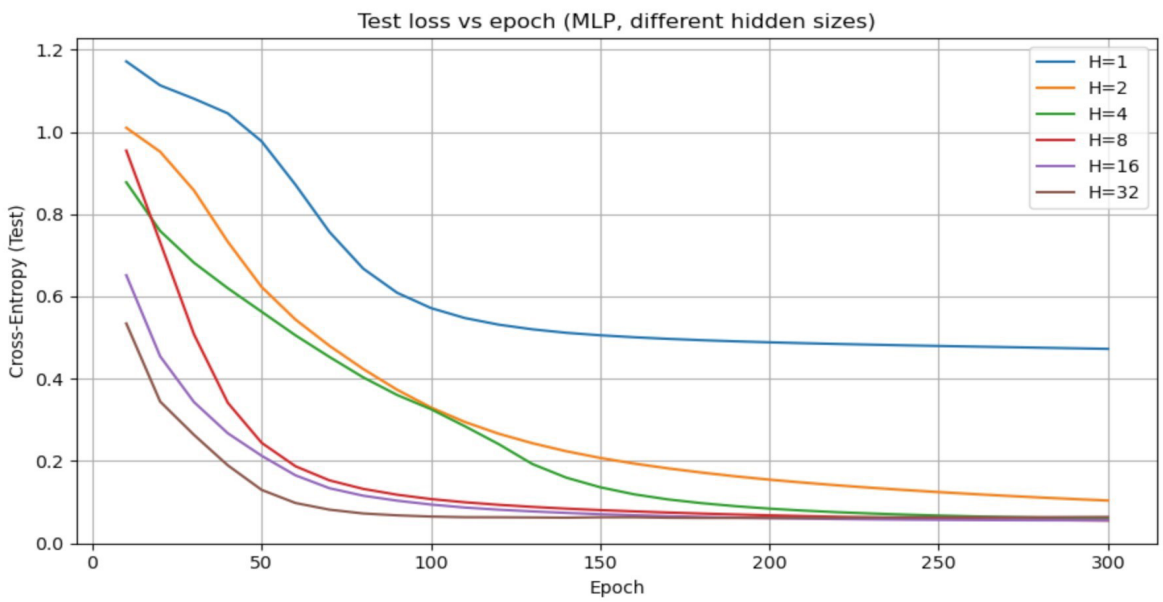
```
def norm_set(x: torch.Tensor, mu: torch.Tensor, sigma: torch.Tensor) ->
torch.Tensor:
    return (x - mu) / sigma

# Compute μ, σ from TRAINING set only
mu    = x_train.mean(dim=0, keepdim=True)
sigma = x_train.std(dim=0, unbiased=False, keepdim=True)  # population std

# Apply to train and test
x_train = norm_set(x_train, mu, sigma)
x_test  = norm_set(x_test,  mu, sigma)

print("Training set mean:", x_train.mean(dim=0))
print("Training set std: ", x_train.std(dim=0, unbiased=False))
```

Built-in MLP & fast loop (sparse eval + early stopping)

```
def make_mlp(H: int) -> nn.Sequential:
    return nn.Sequential(nn.Linear(4, H), nn.ReLU(), nn.Linear(H, 3))
```

hidden	train_acc	test_acc	train_loss	test_loss
8	0.983333	1.000000	0.044525	0.055483
4	0.983333	1.000000	0.047119	0.059684
16	0.983333	1.000000	0.040202	0.055775
32	0.983333	0.966667	0.035337	0.061652
2	0.975000	0.933333	0.071764	0.103958
1	0.725000	0.766667	0.474536	0.472564





Use of AI. I utilized AI tools as supplemental learning for deeper understanding, sometimes for explanation in clearer language, debugging code, and proofreading the report as an ESL student. Core ideas, code implementation, and decisions on analysis remain mine. I validated the outputs, reworded the suggestions in my own words, and made sure that the final submission reflects my understanding and meets the requirements of the assignment.

References:

- Iris dataset: https://scikit-learn.org/stable/datasets/toy_dataset.html#iris-dataset
- Course lecture notes on Neural Networks
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- PyTorch documentation: https://pytorch.org/docs/